

Assignment # 1

Muiz Asif

54791

"Analysis of Algorithms"

Instructor: Mr. Muhammad Usman Shareef

Q1) Algorithm Design and Asymptotic Analysis.

Given an array of n integers, design an algorithm to find largest and smallest elements in array.

1) Algorithmic Strategy:

Linear Scan, is like holding two sticky notes, biggest so far or smallest so far.

- We start by copying $A[0]$ into both notes, As it is the only element seen so far.
- We visit each remaining element once. If it's bigger than max note, replace Max.

6) And if it's smaller than min note, replace Min.

• When the loop ends, notes hold the answer.

Every element is compared with all and nothing is missed, It's for traversing and array sum.

2) Pseudocode 8

Algorithm MINMAX(A, N, Max, Min)

Step-1 [Initialize]

MAX = A[0]

MIN = A[0]

I = 1

Step-2 [Scan Array]

repeat step 3 while $I \leq N-1$

Step-3 [Compare and update]

if $(A[I] > \text{max})$ then

MAX = A[I]

End if

if $(A[I] < \text{min})$ then

Min = A[I]

End if

I = I + 1

Step-4 [Display result]

write (Max, Min)

Step-5 [Finish]

Exit

3

3) Primitive Operations

- Assigning value

$Max = A[0], Min = A[0], i = 1$
 $Max = A[i], Min = A[i], i = i + 1$

- Indexing an Array

$A[0], A[i]$

- Comparing two numbers

$i < N-1, A[i] > Max,$
 $A[i] < Min$

- Arithmetic Operation

$i + 1$

- Output

write (max, min)

4) Best and worst case running time

Before the loop

- $Max = A[0] \rightarrow 1$ index + 1 assignment = 2

- $Min = A[0] \rightarrow 2$

- $i = 1 \rightarrow 1$

- Total = 5

Each loop pass

- loop test = 1

- $A[i] > Max \rightarrow 1$ id: index + 1 comparison = 2

(4)

- $A[I] < \text{Min} \rightarrow 2$
- $I = I + 1 \rightarrow 1 \text{ arithmetic} + 1 \text{ assignment} = 2$
- Total = 7

Best case 8

This happens when all elements are equal

$$T(n) = 5 + 7(n-1) + 1 + 1$$

$$= 7n$$

Worst case 8

An update happens in every pass.

$$T(n) = 7n + 2(n-1)$$

$$= 9n - 2$$

These depend on how we count operations.

The result is in constant times (n).

$$T(n) = cn$$

5) Asymptotic Notation 8

- Best case = $7n = \Theta(n)$
- Worst case = $9n - 2 = \Theta(n)$

- $O(g(n)) : T(n) \leq c \cdot g(n)$ (upper bound)
- $\Omega(g(n)) : T(n) \geq c \cdot g(n)$ (lower bound)
- $\Theta(g(n))$: both at same time

We check 8

$$7n \leq T(n) \leq 9n \text{ for all } n \geq 1 \text{ so}$$

$$c_1 = 7$$

$$c_2 = 9$$

$$n_0 = 1$$

(5)

6) Space Complexity :

Using formula : $C(P) = C + S$
 • fixed :

Max, Min, I, N, which are constant number of variables.

• Variable part :

The array A is the input space. It is passed by reference, like no extra copy.

• Additional :

Space : Constant
 Space complexity = $O(1)$ extra space.



Q2) Design and Compare Algorithms :

Given n integers, design an algorithm to determine whether a particular value exists in array or not.

1) Two Strategies :

1) linear Search :

Check $A[0]$, $A[1]$, $A[2]$ one by one and stop when X is found.

This works on one array either sorted or not.

⑥

2) Binary Search

This needs a sorted array.

- looks at the middle element.
- If it equals, we found X.
- If X is smaller, ignore the right side.
- If X is bigger, ignore the left half.
- Repeat what's left.

2) Pseudocode:

1) Linear

Algorithm [Search (A, N, x, I)]

Step-1 [initialize]

$I = 0$

Step-2 [Scan]

repeat step-3 while $I \leq n-1$

Step-3 [compare]

if ($A[I] = x$) then

return (I)

End if

$I = I + 1$

Step-4 [Not Found]

return (-1)

2) Binary :

⑦

Algorithm : BSearch (A, N, X, low, high, mid)

Step-1 [Initialize]

low = 0

high = N-1

Step-2 [repeat while search area is not empty]
repeat step-3 while low <= high.

Step-3 [check the middle]

mid = (low + high) / 2

if (A[mid] = X) then

return (mid)

Else if (A[mid] < X) then

low = mid + 1

Else

high = mid - 1

End if

Step-4 [Not found]

return (-1)

Best and worst case Analysis :

Linear :

o Best case : X is the first element so I check.

$T(n) = C$, best case = 1

o Worst case : X is the last element or isn't there. All are checked

$T(n) = n \cdot C + C$

worst case.

2) Binary %

⑧

- Best case % x is exactly at the middle position

$$T(n) = C$$

- Worst case %

x is not found or found at end,

$$n \rightarrow n/2, n/4 \dots 1$$

worst case.

Assume $n = 2^k$, then $k = \log_2 n$ are needed, so $\log_2 n + 1$ passes each doing constant work.

$$T(n) = C \cdot \log_2 n + C$$

4) Big - O, Ω , Θ

Algo	B	W	O	n	Θ
Linear	$\Theta(1)$	$\Theta(n)$	$O(n)$	$\Omega(1)$	$\Theta(1), \Theta(n)$
Binary	$\Theta(1)$	$\Theta(\log n)$	$O(\log n)$	$\Omega(1)$	$\Theta(1), \Theta(\log n)$

Space % both use $O(1)$ extra space.

5) Comparison

Point	L. Search	B. Search
Sorted data?	No	Yes
Best case	$O(1)$	$O(1)$
Worst case	$O(n)$	$O(\log n)$
Extra Space	$O(1)$	$O(1)$
Complexity	Simple	Tricky

Which is efficient?

- Sorted data: Binary as it is fast
- One Search: linear as sorting costs more.
- Small n : linear search.

Both are good, it all depends on us or the requirements.

6) Effect of input size and arrangement.

- Size:
 - if Small (n), both Algos finish instantly, but linear is simple.
 - if large (n), linear needs upto n Searches or steps, but binary needs only $\log_2 n$
- Fast growing function takes more time, whatever the constants.

(10)

Arrangement:

- Sorted & Use Binary Search, comparing X with middle and dividing.
- Un-Sorted & Binary search gives wrong answers, cuz X could be in half we discard.
- We use binary search for large and sorted data.
- We use linear search for small, unsorted or for single search data.

